

Travail de Bachelor

Pipeline distribué d'annotation automatique d'images géospatiales

Ray et Kubernetes au service de la segmentation GPU

Étudiant	Dani Tiago Faria dos Santos
Superviseure	Prof. Bertil Chapuis
Département	Technologies de l'information et de la communication (TIC)
Filière	Informatique et systèmes de communication (ISC)
Orientation	Réseaux et systèmes (ISC-RS)
Entreprise mandante	Prof. Bertil Chapuis HEIG-VD — Institut IICT Route de Cheseaux 1 1400 Yverdon-les-Bains
Année académique	2025-26

Yverdon-les-Bains, le 01.05.2026

Table des matières

Authentification	5
Préambule	6
Resumé	7
1 Introduction	8
1.1 Contexte	8
1.2 Problème	8
1.3 Objectifs	9
2 État de l'art	10
2.1 Segmentation d'images	10
2.2 Calcul distribué : Ray	11
2.3 Stockage objet	13
2.4 Format de résultats	13
2.5 Annotation	15
2.6 Observabilité	16
3 Exemple de Chapitre	17
3.1 Rédiger des expression mathématiques	17
3.2 Insérer des images	17
3.3 Créer et citer une référence bibliographique	18
3.4 Créer une référence à une autre partie du document	18
3.5 Afficher une commande simple ou du bash	18
3.6 Inclure du code	19
4 Architecture	20
4.1 Vue d'ensemble	20

4.2 Infrastructure Kubernetes	20
4.3 RayCluster	21
4.4 Cache du modèle SAM3	21
4.5 Stratégie de tuilage	21
4.6 Format de sortie Parquet	21
4.7 Intégration Label Studio	22
5 Implémentation	23
5.1 Image Docker	23
5.2 Pipeline Python	23
5.3 Manifestes Kubernetes	25
5.4 Conversion vers Label Studio	26
6 Résultats	28
7 Conclusion	29
Bibliographie	30
Annexes	33
I Glossaire	34
II Outils utilisés	36
III Journal de travail	38

Authentification

Par la présente, j'atteste avoir réalisé ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Dani Tiago Faria dos Santos

Yverdon-les-Bains, le 01.05.2026

Préambule

Ce travail de Bachelor (ci-après TB) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'Ecole.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD

Le Chef de département TIC

Yverdon-les-Bains, le 01.05.2026

Resumé

Travail de Bachelor 2025-26

Titre: Pipeline distribué d'annotation automatique d'images géospatiales

Sous-titre: Ray et Kubernetes au service de la segmentation GPU

Ce travail porte sur la conception et le déploiement d'un pipeline distribué d'annotation automatique d'images géospatiales. Le pipeline exploite SAM3 pour la segmentation et Ray pour distribuer le traitement sur les GPUs du cluster Kubernetes de la HEIG-VD. Les images sont lues depuis le service S3 sur MinIO, découpées en patches et traitées en parallèle par des workers Ray. Les métadonnées GPS extraites de l'EXIF sont associées aux polygones produits, stockés au format Parquet sur S3. Une couche d'observabilité basée sur Prometheus, Loki et Grafana permet de surveiller l'état du cluster, ses performances et d'identifier les incidents.

Étudiant	Dani Tiago Faria dos Santos
Superviseure	Prof. Bertil Chapuis
Entreprise mandante	Prof. Bertil Chapuis

1 Introduction

1.1 Contexte

L’Institut IICT de la HEIG-VD conduit le projet NearAI, dont l’objectif est de construire une base de données géospatiale d’éléments routiers à partir d’acquisitions mobiles. Les images sont capturées par des véhicules équipé de systèmes différents comme par exemple un Trimble MX50 ou d’une goPro Max, qui produisent des panoramas équirectangulaires allant jusqu’à 8’192 x 4’096 pixels accompagnés de coordonnées GPS enregistrées dans l’EXIF. Le jeu de données cible actuel dépasse 300’000 images.

Annoter manuellement ce volume n’est pas faisable, un annotateur humain habile et expérimenté consacrant trente secondes par image mettrait plus de 2’500 heures pour couvrir l’ensemble du corpus. L’annotation automatisée est la seule voie viable.

1.2 Problème

Plusieurs classes d’objets sont ciblées, comme par exemple, les panneaux de signalisation (`sign`) ou encore les marquages au sol (`road_marking`). Pour chaque image, le pipeline doit produire un ensemble de polygones identifiant ces objets, avec, leur classe, leur score de confiance, leurs coordonnées GPS et leurs dimensions normalisées.

Ces polygones transitent ensuite vers Label Studio (ou toute autre plateforme de traitement d’image comme celle développée par mon collègue Valentin Ricard), où, des annotateurs humains corrigent et valident les prédictions.

Chaque image à 8 192 x 4 096 pixels dépasse la fenêtre d’entrée de tout modèle de segmentation courant. Il faut donc découper l’image en tuiles avant l’inférence. Ensuite, traiter 300’000 images en un délai raisonnable impose de distribuer le calcul sur plusieurs GPUs en parallèle.

1.3 Objectifs

1.3.1 Traitement des images

Ce travail conçoit et déploie un pipeline distribué couvrant les étapes suivantes :

1. Lecture des images depuis le bucket S3.
2. Découpage en tuiles 512 x 512 pixels et inférence sur chaque tuile.
3. Extraction des polygones, normalisation des coordonnées et association des métadonnées GPS issues de l'EXIF.
4. Écriture des résultats au format Parquet sur le bucket.
5. Import des pré-annotations dans Label Studio pour validation humaine.

Tout cela s'exécute sur le cluster Kubernetes de la HEIG-VD via le framework Ray, qui distribue les tâches GPU sur les workers disponibles.

1.3.2 Côté utilisateur

Une API servira de porte d'accès aux développeurs ou utilisateurs du service pour faciliter l'accès aux services batch ou on-demand.

1.3.3 Analyse de la pipeline

La pipeline sera annaylable au niveau de ses performances et état via un dashboards alimenté de métriques, logs et traces.

2 État de l'art

2.1 Segmentation d'images

2.1.1 SAM3

SAM3 (Segment Anything Model 3) est le modèle de segmentation publié par Meta en 2024 [1]. Il hérite de SAM2 et étend ses capacités aux images de haute résolution et à la vidéo. SAM3 adopte une architecture Vision Transformer (ViT) comme encodeur d'image et un décodeur léger qui produit des masques binaires à partir de prompts géométriques (points, boîtes, polygones).

Le modèle fonctionne en mode *promptable* et en mode *everything mode*.

En mode **prompt**, il attend simplement un message comme : *Un panneau octogonal avec le texte STOP à son centre.*

En mode **everything**, il segmente tous les objets détectables de l'image. Aucune supervision n'est fournie à l'inférence et SAM3 propose des masques candidats que le pipeline filtre par classe et score.

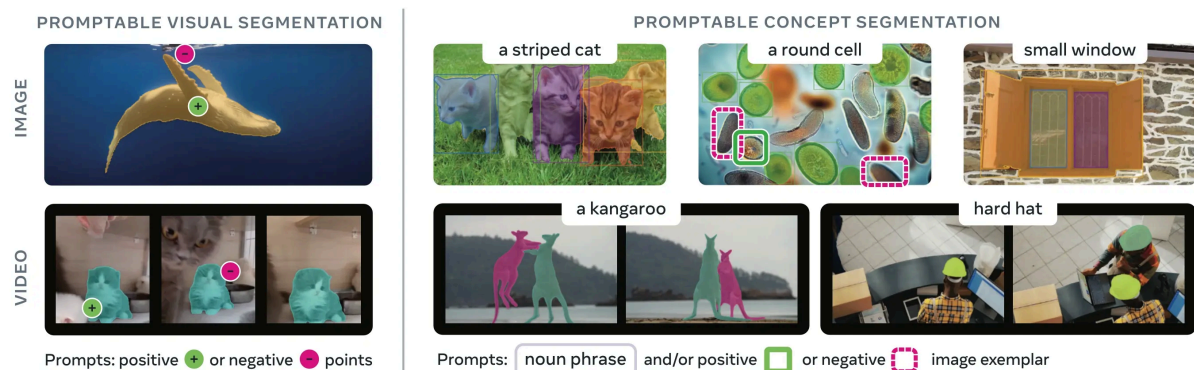


Fig. 1. – Exemple du mode prompt de SAM3. Source : <https://docs.ultralytics.com/fr/models/sam-3/>

SAM3 a été entraîné sur SA-1B, un corpus de 1,1 milliard de masques sur 11 millions d'images. Cette couverture lui confère une généralisation forte sur des domaines non vus à l'entraînement, dont les images routières équirectangulaires.

2.1.2 Fenêtre d'entrée et stratégie de tuilage

SAM3 accepte des images jusqu'à 1'024 x 1'024 pixels. Une panoramique de 8'192 x 4'096 pixels doit donc être découpée avant l'inférence. Ce travail adopte des tuiles de 512 x 512 pixels, ce qui produit 128 tuiles par image à pleine résolution. Un downsampling à 50 % ramène ce nombre à 32 tuiles et réduit le temps d'inférence d'un facteur 4, au prix d'une perte de détail acceptable pour les classes cibles.

NB : SAM3 redimensionne automatiquement grandes les images sur une résolution autour de 1024px, donc il n'y pas d'intérêt d'aller au-delà.

2.1.3 Distorsion équirectangulaire

Les images équirectangulaires présentent une distorsion géométrique croissante vers le zénith et le nadir. Les objets cibles (panneaux, marquages) se concentrent dans la bande centrale de l'image, correspondant à $\pm 30^\circ$ d'élévation, là où la distorsion est minimale. La correction de projection n'est donc pas implémentée, elle apporterait un gain marginal pour un coût d'implémentation élevé. Le bas du panorama est en grande partie occulté par la carrosserie du véhicule. Ce choix est documenté comme limitation connue.

2.2 Calcul distribué : Ray

Ray est un framework Python open-source pour distribuer des workloads ML/IA sur des clusters hétérogènes CPU/GPU [2]. Il expose deux primitives fondamentales.

Une `task` (`@ray.remote` sur une fonction) est une unité de calcul sans état, exécutée de façon asynchrone sur un worker disponible.

Un `Actor` (`@ray.remote` sur une classe) est une unité de calcul avec état, maintenu en mémoire sur un worker assigné. L'Actor est LE patron adapté au chargement de modèles. Le modèle est chargé une fois dans `__init__`, puis réutilisé sur toutes les requêtes sans rechargement, évitant les erreurs de mémoire insuffisante (OOM) qui surviennent lorsqu'un modèle est rechargé à chaque tâche.

```

1 @ray.remote(num_gpus=1)
2 class SAM3Worker:
3     def __init__(self):
4         self.model = load_sam3() # chargé une fois
5
6     def process(self, batch):
7         return infer(self.model, batch)

```

python

Ray gère l'allocation des ressources (CPU, GPU, mémoire), la sérialisation des données entre driver et workers via son object store partagé, et la récupération sur défaillance d'un worker.

2.2.1 KubeRay

KubeRay est l'opérateur Kubernetes officiel pour Ray [3]. Il introduit la ressource **RayCluster** (CRD `ray.io/v1`), qui déclare un nœud head et un ou plusieurs groupes de workers. L'opérateur crée et gère les pods correspondants, expose les ports GCS (6379), dashboard (8265), métriques (8080) et client Ray (10001) via des Services Kubernetes.

Le driver externe se connecte au cluster via `ray.init("ray://ray-cluster-head-svc:10001")`.

Les workers ne sont pas contactés directement car Ray dispatche les tâches via le Global Control Store (GCS) hébergé sur le head.

Une distinction importante sépare la terminologie Kubernetes de la terminologie Ray :

Terme	Sens Kubernetes	Sens Ray
Head	Pas d'équivalence directe	Nœud unique hébergeant le GCS, le scheduler et le dashboard
Worker	Pod dans un Deployment	Nœud Ray enregistré auprès du GCS, exécute les tâches
Service	ClusterIP stable pour la sélection de pods	Expose GCS (6379), dashboard (8265), client (10001)

Tableau 1. – Terminologie Kubernetes vs Ray

2.3 Stockage objet

2.3.1 MinIO

MinIO est un serveur de stockage objet compatible avec le protocole S3 [4]. La HEIG-VD l'exploite sur un NAS Synology SA3200D, installé via Container Manager avec l'image officielle minio/minio. Le pipeline accède à MinIO via `boto3` avec les variables d'environnement S3 standard (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_ENDPOINT_URL`).

2.3.2 S3

S3 est un protocole, pas un produit. L'endpoint peut être remplacé sans modifier le code du pipeline.

2.3.3 Alternatives

Des alternatives comme RustFS (Apache 2.0, moins d'un an) ou CEPH (mature, LGPL, bloc/fichier/objet) ont été évaluées. La décision est de conserver MinIO car la migration ajouterait du risque sans bénéfice immédiat, et la configuration S3 est le seul point à modifier si un changement de backend devient nécessaire.

2.4 Format de résultats

2.4.1 Parquet

Parquet est un format binaire orienté colonnes conçu pour les workloads analytiques [5]. Chaque fichier est découpé en *row groups* eux-mêmes découpés en *column chunks*. Cette organisation permet de lire uniquement les colonnes nécessaires à une requête sans charger l'ensemble du fichier.

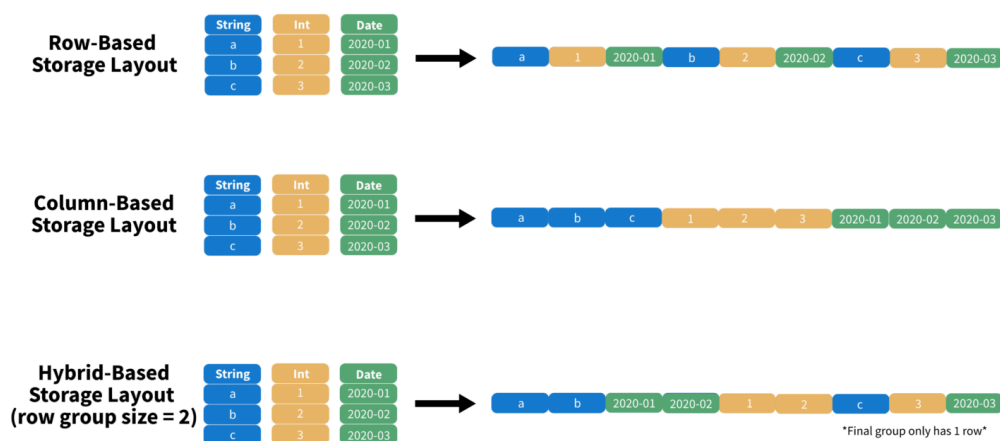


Fig. 2. – Parquet exploite une solution hybride pour obtenir de meilleures performances. Source : <https://towardsdatascience.com/demystifying-the-parquet-file-format-13adb0206705/>

Parquet stocke des statistiques min/max par column chunk. Un moteur de requête peut ainsi ignorer des row groups entiers si le prédicat tombe hors de leur plage. C'est le *predicate pushdown*, supporté nativement par PyArrow. Filtrer les polygones par zone GPS ou par seuil de score ne charge que les colonnes `latitude`, `longitude` et `score`, indépendamment du volume total.

2.4.2 PostGIS

PostGIS (extension PostgreSQL pour les données géospatiales) a été évalué comme alternative. Il offre des index spatiaux GIST et des requêtes géométriques natives (`ST_Within`, `ST_Intersects`). La décision est de le remplacer par Parquet sur S3. Les requêtes du projet ne nécessitent pas de jointures géospatiales complexes, et Parquet évite de maintenir une base de données.

2.4.3 JSON

JSON (JavaScript Object Notation) est le format accepté par labelstudio pour importer les données géospatiales sur une images. Ce format va être utiliser uniquement pour le mode `on demand` et en `legacy` pour labelstudio afin de visualiser les résultats des runs.

```
1  [{
2    "data": {
3      "image": "s3://nearai/data/acquisitions/.../image.jpg"
4    },
5    "predictions": [{
6      "model_version": "SAM3",
7      "result": [{
8        "type": "polygonlabels",
9        "from_name": "label",
10       "to_name": "image",
11       "original_width": 8192,
12       "original_height": 4096,
13       "value": {
14         "closed": true,
15         "polygonlabels": ["sign"],
16         "points": [[42.05, 40.72], ...]
17       }
18     }]
19   }]
20 }]
```

json

2.5 Annotation

2.5.1 Label Studio

Label Studio est une plateforme d'annotation open-source [6]. Elle supporte les polygones (`polygonlabels`) sur images, le stockage S3 comme source d'images, et l'import de pré-annotations via son API REST. Le pipeline produit des pré-annotations SAM3 que des annotateurs humains corrigent et valident dans Label Studio.

Label Studio est configuré avec MinIO comme *cloud storage* source. Les URLs `s3://nearai/...` sont converties en URLs HTTP temporaires signées, ce qui évite d'exposer les credentials de stockage aux navigateurs clients.

2.6 Observabilité

2.6.1 Grafana

Gra

2.6.1.1 Loki

2.6.1.2 Promtail

2.6.2 Prometheus

2.6.3 DCGM

3 Exemple de Chapitre

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facere et urbane Stoicos irridere, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

3.1 Rédiger des expressions mathématiques

Voici des maths intégrés: $\frac{3^2}{4^3}$

ou alors à la ligne:

$$\frac{3x_a}{y_b^2 + 4}$$

3.2 Insérer des images

Voici comment mettre une image.

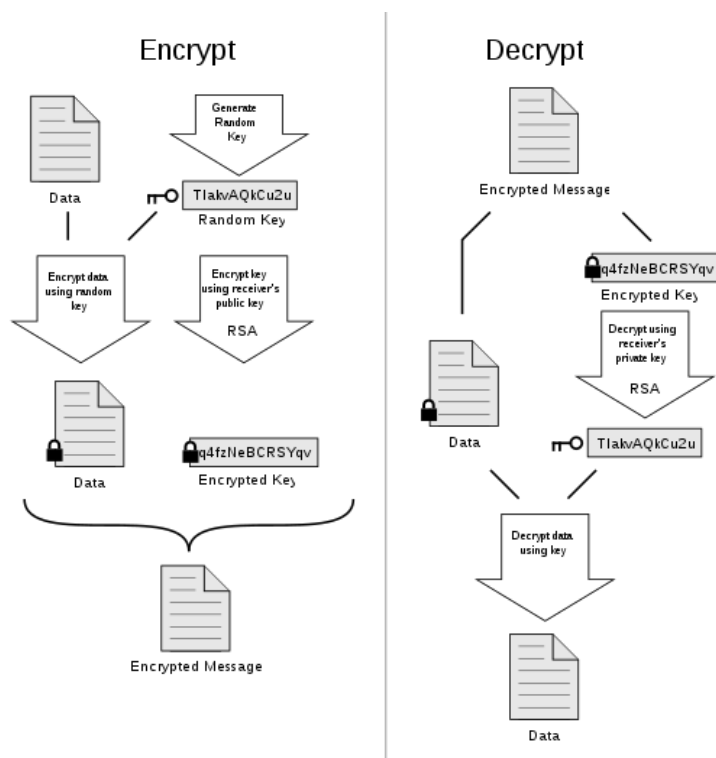


Fig. 3. – Schéma PGP

3.3 Créer et citer une référence bibliographique

Ceci est un exemple de citation d'un livre de Pasini [7]

Mais aussi le site de Black Alps 2019 [8]

3.4 Créer une référence à une autre partie du document

On peut aussi ajouter une référence à la section 3.6

On peut aussi ajouter une référence à l'introduction, Chapitre 1.

Comme montre la Fig. 3, on peut référencer une figure.

3.5 Afficher une commande simple ou du bash

Exemple : Test d'une commande bash shell `ls` :

```
1 $> ls -al test_underscore $$* "coucou"
```

sh

3.6 Inclure du code

```
1 #include <stdio.h>
2 int main(int argc, char* argv[])
3 {
4     printf("Hello World!\n");
5     return 0;
6 }
```

C

4 Architecture

4.1 Vue d'ensemble

Le pipeline suit un flux linéaire en cinq étapes :

1. **Lecture** : le driver liste les objets du préfixe S3 d'entrée et distribue les clés d'image aux workers Ray.
2. **Téléchargement** : chaque worker télécharge l'image depuis MinIO et extrait les coordonnées GPS de l'EXIF.
3. **Inférence** : l'image est découpée en tuiles 512×512 px (après downsampling optionnel), chaque tuile est passée à SAM3 en mode *everything*.
4. **Agrégation** : les masques produits sont convertis en polygones, normalisés en pourcentage des dimensions de l'image originale, filtrés par score de confiance.
5. **Écriture** : les polygones sont sérialisés dans un fichier Parquet et envoyés sur MinIO.

4.2 Infrastructure Kubernetes

Le pipeline s'exécute sur le cluster `iict-rad` de la HEIG-VD (Kubernetes 1.32.5). Le cluster expose 9 GPUs répartis sur trois nœuds :

Nœud	GPUs	Modèle	VRAM
<code>iict-suchet</code>	3	NVIDIA L40S	46 GB
<code>iict-k8s-node4-rad</code>	2	NVIDIA A40	46 GB
<code>iict-chasseron</code>	4	NVIDIA L4	23 GB

Tableau 2. – GPUs disponibles sur le cluster `iict-rad`

Les workers Ray ciblent exclusivement les nœuds L40S et A40 via `nodeAffinity`. Le nœud `iict-chasseron` est exclu pour deux raisons : ses L4 offrent une puissance de calcul inférieure (23 GB VRAM vs 46 GB), et il portait un taint `node.kubernetes.io/disk-pressure` lors des tests.

Ce taint, appliqué automatiquement par Kubernetes quand l’usage disque franchit un seuil, expose les pods à des évictions SIGTERM sans préavis.

Le GPU Operator NVIDIA est installé sur le cluster (confirmé via le wiki IICT). Il installe automatiquement les drivers GPU, le device plugin et DCGM Exporter sur chaque nœud. Les requêtes de ressource `nvidia.com/gpu` fonctionnent sans configuration manuelle.

4.3 RayCluster

4.3.1 Injection des credentials

4.4 Cache du modèle SAM3

4.5 Stratégie de tuilage

4.6 Format de sortie Parquet

Chaque image produit un fichier Parquet nommé `<acquisition_id>/<image_stem>.parquet`, écrit sur le préfixe S3 de sortie. Le schéma est le suivant :

Colonne	Type	Description
<code>image_key</code>	string	Chemin S3 complet de l’image source
<code>acquisition_id</code>	string	Dossier parent (identifiant de l’acquisition)
<code>label</code>	string	sign ou road_marking
<code>score</code>	float32	Score de confiance SAM3 (0–1)
<code>points</code>	string	Polygone JSON encodé, en % des dimensions de l’image
<code>original_width</code>	int32	Largeur de l’image source en pixels
<code>original_height</code>	int32	Hauteur de l’image source en pixels
<code>latitude</code>	float64	Latitude GPS en degrés décimaux (null si absent)
<code>longitude</code>	float64	Longitude GPS en degrés décimaux (null si absent)

Tableau 3. – Schéma Parquet de sortie du pipeline

La compression Snappy est appliquée. Les fichiers sont interrogeables directement depuis DuckDB ou PyArrow sans étape de chargement en base de données.

4.7 Intégration Label Studio

Label Studio est connecté à MinIO comme *source storage* S3. Les URLs `s3://nearai/...` sont converties en URLs HTTP temporaires pré-signées, servies directement du stockage au navigateur sans proxy.

L'interface de labeling XML définit deux classes :

```
1 <View>
2   <Image name="image" value="$image"/>
3   <PolygonLabels name="label" toName="image">
4     <Label value="sign" background="#FF0000"/>
5     <Label value="road_marking" background="#FFFF00"/>
6   </PolygonLabels>
7 </View>
```



Les pré-annotations importées doivent utiliser `from_name: "label"` et `to_name: "image"` pour correspondre à cette interface. Une divergence de noms provoque l'affichage des polygones sans label (gris).

5 Implémentation

5.1 Image Docker

Le pipeline s'exécute dans une image Docker basée sur `nvidia/cuda:12.6.3-cudnn-devel-ubuntu24.04`, publiée sur GitHub Container Registry (`ghcr.io/nearai-interreg/ray-sam3:latest`). Les couches principales sont installées dans cet ordre pour maximiser le cache Docker :

1. Python 3.12 dans un venv isolé (`/opt/venv`), évitant les conflits avec les paquets système.
2. PyTorch 2.7.0 compilé pour CUDA 12.6 (`index download.pytorch.org/whl/cu126`).
3. SAM3 cloné depuis le dépôt Meta (`github.com/facebookresearch/sam3`) et installé avec ses extras `notebooks,train,dev`.
4. Ray 2.54.0 avec les extras `data,train,serve,default` et les dépendances pipeline : `exif, Pillow, opencv-python-headless, numpy>=1.26,<2, boto3, scipy, pyarrow`.

5.2 Pipeline Python

Le fichier `sam3_minio_pipeline.py` est le point d'entrée unique. Il supporte deux modes via l'argument `--local` :

- **Mode local** (`--local`) : `ray.init()` — Ray s'initialise sur la machine locale. Le pod doit disposer d'un GPU. Utilisé pour les tests sur un Job K8s à GPU unique.
- **Mode cluster** (défaut) : `ray.init("ray://ray-cluster-head-svc:10001")` — le driver se connecte au RayCluster via le protocole Ray Client et distribue les tâches aux workers distants.

5.2.1 Actor SAM3

L'Actor `SAM3Actor` est décoré avec `@ray.remote(num_gpus=1)`. Ray crée une instance d'Actor par GPU disponible dans le cluster et refuse d'en créer davantage si les ressources sont épuisées. Le modèle est chargé une seule fois dans `__init__` via HuggingFace Hub et réutilisé sur toutes les requêtes.

Un bug subtil a été identifié lors des tests en mode local : l'utilisation de `.options(num_gpus=0)` pour modifier l'allocation au moment de la création écrasait silencieusement la déclaration du décorateur et forçait `CUDA_VISIBLE_DEVICES=""`, rendant CUDA invisible au processus. L'erreur résultante était :

```
1 RuntimeError: No CUDA GPUs are available
```

La correction consiste à supprimer l'appel `.options()` et laisser le décorateur gérer l'allocation dans les deux modes.

5.2.2 Extraction GPS depuis l'EXIF

Les coordonnées GPS sont extraites de l'EXIF de chaque image au moment du téléchargement, via la bibliothèque `exif`. Le format EXIF stocke les coordonnées en degrés/minutes/secondes (DMS) avec une référence cardinale. La conversion en degrés décimaux suit :

$$d_{\text{decimal}} = d + \frac{m}{60} + \frac{s}{3600}$$

La référence cardinale (N/S, E/O) détermine le signe du résultat. Les images sans EXIF GPS stockent `null` dans les colonnes `latitude` et `longitude` du Parquet. Les erreurs de parsing EXIF sont silencieusement ignorées pour ne pas interrompre le traitement de l'image.

5.2.3 Distribution des tâches

Le driver récupère la liste des clés S3 du préfixe d'entrée et la divise en batches de taille `--batch_size` (défaut : 4 images). Chaque batch est soumis à un worker disponible via `.process.remote(batch)`. Le driver attend tous les futurs avec `ray.get(futures)` avant de passer au batch suivant.

Ce schéma simple évite la surcharge d'un scheduling dynamique complexe. Le nombre de workers actifs est borné par le nombre de GPUs disponibles dans le cluster.

5.2.4 Reprise sur interruption (`--resume`)

L'argument `--resume` filtre les images dont un fichier Parquet existe déjà à la destination. Avant de distribuer le travail, le driver appelle `list_objects_v2` sur le préfixe de sortie et construit un ensemble des clés déjà traitées. Ce mécanisme permet de reprendre un run interrompu sans retraiter les images déjà traitées.

5.2.5 Résumé de session

À la fin de chaque exécution, le pipeline affiche un résumé sur `stdout` :


```
1 Done: 40 images, 2230 detections – 111.0s/image (total 4440s)
```

Ce résumé est capté par Promtail (futur déploiement observabilité) et permet de vérifier le débit sans interroger les fichiers Parquet.

5.3 Manifestes Kubernetes

Trois manifestes couvrent les scénarios de déploiement.

5.3.1 RayCluster (deploy/ray/rayCluster.yaml)

Déclare le cluster Ray permanent : 1 head (2 CPU, 4 Gi) et jusqu'à 3 workers GPU (1 GPU, 8 CPU, 32 Gi). Le `nodeAffinity` préfère les L40S (poids 100) aux A40 (poids 50). Les credentials MinIO (`minio-secret`) et le token HuggingFace (`hf-secret`) sont injectés via `secretKeyRef` dans le spec worker — pas dans le head, qui n'exécute aucune tâche GPU.

5.3.2 Job driver cluster (deploy/ray/job-sam3-driver.yaml)

Job Kubernetes sans GPU qui se connecte au RayCluster et orchestre le traitement d'un préfixe S3. Il se termine dès que l'ensemble des images est traité. `ttlSecondsAfterFinished: 3600` supprime le Job automatiquement après une heure.

Arguments typiques :

```
1  args:
2    - --s3_uri
3    - s3://nearai/data/acquisitions/Samples/01_images/
4    - --s3_output_uri
5    - s3://nearai/dani/predictions/
6    - --labels
7    - sign,road_marking
8    - --batch_size
9    - "4"
10   - --num_workers
11   - "3"
```

yaml

5.3.3 Job test local (tests/RAY/job-sam3-ray-test.yaml)

Job Kubernetes avec 1 GPU, utilisé pour valider le pipeline sans cluster Ray (`--local --num_workers 1`). Il monte le PVC HuggingFace à `/root/.cache/huggingface` et un volume `emptyDir` de 16 Gi en mémoire sur `/dev/shm` pour accélérer le partage de tenseurs entre les processus PyTorch. `hostIPC: true` est activé pour permettre la communication inter-processus via la mémoire partagée du nœud.

5.3.4 PVC cache HuggingFace (tests/RAY/pvc-hf-cache.yaml)

PVC Longhorn de 10 Gi en mode `ReadWriteOnce`. Monté sur le pod worker à `/root/.cache/huggingface`. Le volume reste `Bound` entre les redéploiements, évitant le re-téléchargement des 3,3 GB du modèle SAM3 à chaque run.

5.4 Conversion vers Label Studio

Les fichiers Parquet produits par le pipeline sont convertis en JSON Label Studio pour l'import de pré-annotations. Le format attendu par Label Studio est :

```
1  [{
2    "data": {
3      "image": "s3://nearai/data/acquisitions/.../image.jpg"
4    },
5    "predictions": [{
6      "model_version": "SAM3",
7      "result": [{
8        "type": "polygonlabels",
9        "from_name": "label",
10       "to_name": "image",
11       "original_width": 8192,
12       "original_height": 4096,
13       "value": {
14         "closed": true,
15         "polygonlabels": ["sign"],
16         "points": [[42.05, 40.72], ...]
17       }
18     }]
19   }]
```

[json](#)

```
20 }]
```

Deux contraintes sont critiques :

- Le tableau de tâches doit être encapsulé dans un objet racine de type liste (pas un objet seul).
- `from_name` doit correspondre exactement au `name` du tag `<PolygonLabels>` dans l'interface XML du projet Label Studio. Toute divergence provoque un import silencieusement invalide (polygones gris).

La prochaine étape est d'automatiser cette conversion et l'appel à l'API REST Label Studio directement depuis le pipeline, conformément à la section 6.4 du cahier des charges.

6 Résultats

7 Conclusion

Bibliographie

- [1] Nikhila Ravi *et al.*, « SAM 2: Segment Anything in Images and Videos », 2024. [En ligne]. Disponible sur: <https://arxiv.org/abs/2408.00714>
- [2] Philipp Moritz *et al.*, « Ray: A Distributed Framework for Emerging AI Applications », 2018. [En ligne]. Disponible sur: <https://arxiv.org/abs/1712.05889>
- [3] Ray Project, « KubeRay: Kubernetes Operator for Ray ». [En ligne]. Disponible sur: <https://ray-project.github.io/kuberay/>
- [4] MinIO Inc., « MinIO High-Performance Object Storage ». [En ligne]. Disponible sur: <https://min.io/docs/minio/linux/index.html>
- [5] Apache Software Foundation, « Apache Parquet ». [En ligne]. Disponible sur: <https://parquet.apache.org/docs/>
- [6] HumanSignal, « Label Studio: Data Labeling Software ». [En ligne]. Disponible sur: <https://labelstud.io/>
- [7] Gildas Avoine, Pascal Junod, Philippe Oechslin, et and Sylvain Pasini, *Sécurité informatique, cours et exercices corrigés*. Vuibert, 2015.
- [8] Sylvain Pasini, « Black Alps 2019 ». [En ligne]. Disponible sur: <https://www.blackalps.ch/ba-19/>

Table des figures

Fig. 1	Exemple du mode prompt de SAM3. Source : https://docs.ultralytics.com/fr/models/sam-3/	10
Fig. 2	Parquet exploite une solution hybride pour obtenir de meilleur performances. Source : https://towardsdatascience.com/demystifying-the-parquet-file-format-13adb0206705/	13
Fig. 3	Schéma PGP	18

Liste des tableaux

Tableau 1 Terminologie Kubernetes vs Ray	12
Tableau 2 GPUs disponibles sur le cluster iict-rad	20
Tableau 3 Schéma Parquet de sortie du pipeline	21
Tableau 4 Glossaire	34
Tableau 5 Journal de travail	38

Annexes

I Glossaire

Terme	Définition
Kubernetes	Système d'orchestration de conteneurs open-source
KubeRay	Opérateur Kubernetes pour déployer des clusters Ray
Ray	Framework Python de calcul distribué, optimisé pour les workloads GPU
SAM3	Segment Anything Model v3 : modèle de segmentation d'images de Meta AI
ZARR	Format N-dimensionnel orienté chunking, évalué comme optimisation optionnelle
EXIF	Métadonnées embarquées dans les fichiers image, contenant notamment les coordonnées GPS
MinIO	Serveur de stockage objet compatible S3
Parquet	Format de stockage colonnaire optimisé pour les requêtes analytiques
Label Studio	Plateforme open-source d'annotation de données pour le machine learning
Pod	Unité de déploiement atomique dans Kubernetes
S3	Protocole de stockage objet défini par Amazon Web Services
GPU	Graphics Processing Unit
Actor	Abstraction Ray permettant de charger un modèle une fois et de le réutiliser sur plusieurs tâches
Prometheus	Système de collecte de métriques par scraping, modèle pull
DCGM Exporter	Exporteur NVIDIA exposant les métriques GPU vers Prometheus
Promtail	Agent de collecte de logs, tourne sur chaque node K8s

Terme	Définition
Loki	Agrégateur de logs compatible S3, intégré à Grafana
Grafana	Interface de visualisation des métriques et des logs

Tableau 4. – Glossaire

II Outils utilisés

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via procedat oratio. Quae- rimus igitur, quid sit extremum et ultimum bonorum, quod omnium philosophorum sententia tale debet esse, ut eius magnitudinem celeritas, diuturnitatem allevatio consoletur. Ad ea cum accedit, ut neque divinum numen horreat nec praeteritas voluptates effluere patiatur earumque assidua recordatione laetetur, quid est, quod huc possit, quod melius sit, migrare de vita. His rebus instructus semper est in voluptate esse aut in armatum hostem impetum fecisse aut in

poetis evolvendis, ut ego et Triarius te hortatore facimus, consumeret, in quibus hoc primum est in quo admirer, cur in gravissimis rebus non delectet eos sermo patrius, cum.

III Journal de travail

Date	Description	Rech. [h]	Dev. [h]	Rapport [h]	Admin [h]
<i>Le journal de travail continue à la page suivante.</i>					
05.03.2026	Kick-off meeting avec le mandant	0	0	0	2
12.03.2026	Recherche sur l'état de l'art	4	0	0	0
19.03.2026	Mise en place de l'environnement de développement et documentation	0	7	1	0
25.03.2026	Rédaction de l'état de l'art	0	0	4	0
25.03.2026	Dev de la partie interface	0	0	4	0
xx.xx.2026	Test	0	0	4	0

Date	Description	Rech. [h]	Dev. [h]	Rapport [h]	Admin [h]
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0

Tableau 5. – Journal de travail